

URP Implementation

I've made only two files, [sender.py](#) and [receiver.py](#)

They are both self contained and can be executed without any other dependencies besides Python's standard library. No makefile or build system is required. I've made multiple classes for each feature of my URP. Notably, PLC, URPSegment and Sender/Receiver to keep things neatly organised.

Key components

URP segment: handles the segment creation, parsing headers as well as providing a checksum calculation

PLC: simulates packet loss and corruption based on the calculations provided in the assignment spec

Sender: runs the main sender logic with sliding window and retransmissions.

Receiver: Main receiver logic with buffering and ensuring in-order delivery.

Some highlights

Threading Model: the sender uses multiple threads, one for receiving ACKS while another for handling data transmission. The receiver uses only one thread.

State machine: I've followed the diagram provided closely. For sender: SYN_SENT -> ESTABLISHED -> FIN_WAIT -> CLOSED. For receiver: LISTEN -> ESTABLISHED -> TIME_WAIT -> CLOSED.

Modular Design: Segment creation/parsing, checksum calculation, and plc emulation are separated into distinct classes for maintainability.

Segment format

Each URP consists of a 6 byte-header as mentioned by the spec, followed by optional payloads:

- Bytes 0-1: Sequence number (16-bit, big-endian)
- Byte 2: Reserved (always 0)
- Byte 3: Flags (3 bits: ACK=0b001, SYN=0b010, FIN=0b100)
- Bytes 4-5: Checksum (16-bit, big-endian)

Sender:

- unacked segments: a dictionary which maps sequence_numbers to tuples of (segment_bytes, payload_length, send_order-counter) and is used for retransmission tracking.
- Send_order_counter: monotonic counter ensuring correct retransmission order ever after seq number wraps around.
- base and next_seq: this is for the sliding window approach, base for the left and next_seq for right edge.

Receiver:

- receive_buffer: a dictionary mapping of sequence numbers to data payloads for out-of-order segment buffering.
- received_seqs: set tracking received sequence numbers to detect dups
- expected_seq: next expected sequence number for cumulative ack generation

All sequence number arithmetic uses modulo 65536 (2^{16}) to handle wrap-around. The seq_compare() function implements wrap-aware comparison for determining if one sequence number is greater than another.

Sender Operation

Connection Establishment

- Generate a random initial sequence number (ISN)
- Creates and sends a SYN segment and track it in our unacked_segments dictionary
- Starts a retransmissions timer. Wait for the SYN_ACK and then retransmit SYN on timeout until acknowledged.

Data Transmission

- The sender reads file data in chunks up to the MSS value
- Creates DATA segments using the seq numbers
- Maintains a sliding window, only sending when window allows (unacked data < max_win)
- Tracks all unacknowledged segments for retransmission
- On receiving an ACK
 - Removes the fully acknowledged segments from unacked segments
 - Updates window base
 - Implements fast retransmit: retransmit oldest unacked segment after 3 duplicate ACKS

Connection Terminations

- On receiving FIN, it writes any remaining buffered data, closes the file and then sends FIN-ACK.
 - Enters TIME_WAIT state for $2 \times \text{MSL}$ seconds to handle delayed segments
 - If you get duplicate FINs, send the ACK again

Buffering:

Receiver maintains a buffer for out-of-order segments within the receive window. When an in-order segment arrives, it writes that segment and any subsequent buffered segments in sequence. Ensuring in-order delivery.

Error detection

Checksum

- IT is implemented using a 16 bit ones complement checksum (<https://cs.newpaltz.edu/~easwarac/CCN/Week5/Checksum.pdf>)
- Formula can be found above
- When we receive the segment, we calculate the checksum as well, compare them, if matches its valid. Otherwise, discard and increment corruption counter.

Error Handling

- Corrupted segments: both sender and receiver will verify checksums, the corrupted ones are discarded and logged as well. The sender discards corrupted ACKS and continues waiting. The receiver discards corrupted DATA/SYN/FIN segments
- Lost segments: handles by timeout based retransmission. Sender tracks all unacked segments and retransmit on timeout.
- Duplicate segments. Tracks received sequence numbers and ignores duplicates but also sends ACKS (leads to duplicate ACK)

Design Trade offs

Threading vs Event loop

- Sender uses threading for ACK while receiver uses a single threaded loop
- Sender needs to send and receive concurrently while receiver only needs to respond to events
- Threading adds complexity but enables concurrent operation

Cumulative ACKs vs Selective acks

- Cumulative acks only
- Simpler implementation and sufficient for requirements
- Less efficient under high loss as itll need to retransmit more data but simpler and more reliable